MILESTONE v1b — ARCHITECTURE & ENGINEERING GUIDE

Objects, Freehand Lasso & Shape Library

Interactive PageObjects • Freehand Ray-Casting Lasso • Cross-Notebook Clipboard • 31

Target: iPadOS 26.5+ (iPad Only) | Repo: github.com/jq-22/notes-app | Release: v1b

1. Executive Summary — Milestone v1b Capabilities

Interactive PageObjects
Basic Shapes & Images

Rectangle, circle, arrow, line, and photo objects with 4-corner resizing, live rotation, move border, and dual end-bar

Freehand Lasso Tool
Ray-Casting Selection

Dotted lasso loop selection for ink strokes and objects. Translate together, recolour with swatches, duplicate, copy,

Cross-Notebook Clipboard
System-Wide Copy & Paste

Copy and paste ink strokes and page objects across sections and across distinct notebooks, preserving vector

Notebook Shape Library
Reusable Diagram Shapes

Save any lassoed ink drawing with a custom label. Stored as raw ink + transparent PNG preview cards. Browse and pull

2. Milestone v1b Layered Architecture & Data Flow

Strict unidirectional dependency: Domain ← Persistence ← Features ← App. Zero third-party dependencies.

Presentation Layer (Features & UI)

`PageEditorView, PencilCanvasScrollView,`

SwiftUI + UIKit interactive views handling PencilKit drawing, gesture translation, handle drags, and toolbars

- •
- • `LassoOverlayView`
- • `ShapeLibraryPopover`

↓

State & ViewModel (LibraryViewModel)

`LibraryViewModel (@Observable @MainActor)`

Coordinates selection, undo/redo triggers, clipboard copy/paste, shape library caching, and debounced persistence

- • `Clipboard Engine`
- • `Shape Lib Manager`
- • `PageObject CRUD`

↓

Storage Boundary (Persistence)

`PageStore Protocol + FilePageStore Implementation`

Atomic file transactions: notebooks.json, Drawings/<id>.pkdrawing, Assets/<id>.dat, ShapeLibrary/<id>.json, Previews/

- • `FilePageStore`
- • `Assets Storage`
- • `ShapeLibrary IO`

↓

Domain Core (Domain Models)

`Notebook, NoteSection, Page, PageObject,`

Pure Swift value types, geometry computations, ray-casting point-in-polygon algorithm, zero UI framework imports

- • `PageObject Geometry`
- • `Lasso Ray-Caster`
- • `ShapeLibraryItem`

Data Integrity & Backward Compatibility Guarantee

Milestone v1a notebook JSON files load seamlessly without migration scripts. The Page decoder defaults missing objects to an empty array [PageObject], ensuring existing student and work notebooks remain 100% intact.

3. PageObject Architecture & Transform Geometry

Unified Handle System

`Corner Dots + Move Border + Rotate Handle`

Rectangles, circles, and images feature 4 circular corner handles (min 44pt touch target) that anchor the diagonally opposite corner during resizing. A top rotation stalk allows

Dual End-Bar Line Handles

`Opposite-End Anchored Scaling`

Lines and Arrows employ an intuitive engineering blueprint design: two perpendicular end-bar handles for resizing and stretching in either direction while pinning the opposing

Vector Basic Shapes

`Rectangle, Circle, Arrow, Line`

Rendered via native SwiftUI vector shapes and CoreGraphics paths, supporting stroke thickness and the 8 Theme ink colour presets with automatic dark mode contrast.

Independent Asset Storage

`Assets/<asset-uuid>.dat Separation`

Inserted photos and images are normalized to PNG data and persisted in a dedicated Assets/ folder referenced by UUID, keeping the main notebooks.json metadata lightweight and

4. Freehand Lasso Ray-Casting & Selection Engine

LassoSelection.swift implements pure geometric point-in-polygon ray-casting to accurately capture ink strokes and

1. Loop Tracing

`LassoSelectionOverlayView`

User draws a freehand loop using finger or pencil. The system records raw points into a closed LassoPolygon, rendering a high-contrast animated dotted stroke.

2. Ray-Casting Enclosure

`LassoPolygon.contains(point:)`

Ray-casting Jordan Curve algorithm casts horizontal rays to test points. An ink stroke is enclosed if ≥60% of its sample points fall inside the polygon; objects are enclosed if their center falls inside.

3. Unified Translation

`ActiveLassoSelection`

Dragging inside the bounding box moves enclosed ink strokes and PageObjects in real time. PencilKit canvas temporarily hides selected strokes to eliminate duplicate rendering.

4. Floating Action Menu

`LassoActionToolbar`

Floating menu provides one-tap actions: Recolour (applying ink swatches to enclosed strokes), Copy, Cut, Convert to Object (freezes strokes as PageObject), Save to Library, and Delete.

5. StrokeGroup Object Transformation & Recolouring

```
// Notes/Domain/PageObject.swift — StrokeGroup Recolouring & Transformation
extension PageObject {
    public func withTintColor(_ newColor: InkColorPreset) -> PageObject {
        guard case .strokeGroup(let data, let w, let h, _) = self.kind,
              let originalDrawing = try? PKDrawing(data: data) else {
            return self
        }
        // Recolor all ink strokes while preserving stroke widths and pressure profiles
        var recoloredStrokes: [PKStroke] = []
        for stroke in originalDrawing.strokes {
            var ink = stroke.ink
            ink.color = newColor.uiColor
            let newStroke = PKStroke(ink: ink, path: stroke.path, transform: stroke.transform, mask:
stroke.mask)
            recoloredStrokes.append(newStroke)
        }
        let updatedDrawing = PKDrawing(strokes: recoloredStrokes)
        let updatedData = updatedDrawing.dataRepresentation()
        var copy = self
```

6. Notebook Shape Library & Cross-Notebook Reuse

Transparent PNG Previews

Saved shapes render transparent preview cards via PKDrawing.image(from:scale:) under

`ShapeLibrary/Previews/<id>.png`

Cross-Notebook Drawer

Tapping the notebook title at the top of ShapeLibraryPopoverView opens a drawer allowing students to pull diagram

`ShapeLibraryPopoverView.swift`

Placing as PageObject

Placing an item creates a centered, resizable, and rotatable .strokeGroup PageObject. Users can reposition and

`placeShapeLibraryItem()`

7. File Storage Layout (Zero Database, Pure Files)

```
Documents/
  ├── notebooks.json                        # Lightweight metadata: hierarchy, PaperTypes, [PageObject]
geometry
  ├── Drawings/<page-uuid>.pkdrawing        # High-density PencilKit binary drawing data
  ├── Assets/<asset-uuid>.dat               # Inserted image and photo binary payloads (PNG/JPEG)
  └── ShapeLibrary/
      ├── <notebook-uuid>.json              # Saved diagram shapes: labels, raw stroke data, bounds
```

8. Automated Verification Matrix — 31 Tests Passing

Verified on iPad Pro 13-inch (M5) Simulator via Xcode 26.6 / iOS 26.5 SDK. 100% test pass rate.

✓ **PageObjectTests (5 tests)** — `NotesTests/Domain/PageObjectTests.swift`

Verifies move/4-corner resize anchoring opposite corner, rotation with rotated hit-testing, line endpoint handle stretching, and backward-compatible v1a JSON decoding.

✓ **LassoSelectionTests (2 tests)** — `NotesTests/Domain/LassoSelectionTests.swift`

Verifies pure ray-casting point-in-polygon math, sample-point threshold enclosure, object center detection, and tiny tap rejection.

✓ **FilePageStoreTests (6 tests)** — `NotesTests/Persistence/FilePageStoreTests.swift`

Verifies notebook round-trip, drawing separation, asset storage lifecycle, shape library JSON persistence, and preview PNG file operations.

✓ **LibraryViewModelTests (9 tests)** — `NotesTests/Features/LibraryViewModelTests.swift`

Verifies PageObject lifecycle, cross-notebook clipboard copy/paste, shape library save/load/place/delete, debounced autosave, and batch move/delete.

✓ **ScribbleClassifierTests (6 tests)** — `NotesTests/Domain/ScribbleClassifierTests.swift`

Verifies synthetic scribble detection over strokes, circle/zigzag non-triggering, and undo grouping.

✓ **NotebookHierarchyTests (3 tests)** — `NotesTests/Domain/NotebookHierarchyTests.swift`

Verifies UUID permanence during reordering, paper overrides, and title sanitization.

Roadmap: Milestone v1c — Structure (Next Up)

• Tables: Set grid (user-configured rows & columns) rendered on the paper layer. Freehand handwriting inside cells without ink clipping. Dynamic row/column resizing.